DOMAIN_NOTES (Phase 0) — Failure Modes, Strategy Tree, Architecture, Cost

This project is not “OCR a PDF”. It is an **escalation-gated, provenance-preserving** extraction pipeline that turns enterprise documents into structured, auditable data.

---

1. Failure Modes Mapped to Corpus Classes

Below, each **failure mode** is tied directly to the four document classes and their representative corpus examples.

1.1 Structure Collapse

- **Class A – CBE Annual Report 2023–24 (native, multi-column + tables)**
  - **Symptom**: Naive `pdfplumber.extract_text()` flattens two-column pages and breaks financial statement tables into line-wrapped strings.
  - **Root cause**:
    - The PDF exposes a text stream with per-character coordinates, but naïve extraction ignores spatial grouping.
    - Column boundaries (distinct x-position modes) and table gridlines are lost, so header–cell alignment disappears.
  - **Impact**: “Total assets” values are separated from their labels; queries like “What is total assets in 2024?” either fail or hallucinate because the model sees ambiguous numeric sequences without a stable header relationship.
- **Class C – FTA Technical Assessment Report (mixed narrative + tables + findings)**
  - **Symptom**: Complex survey result tables spanning multiple pages are broken into partial row fragments, with page footers/headers interleaved in the text flow.
  - **Root cause**:
    - Naive extraction processes pages independently, losing cross-page table continuation structure.
    - Repeated running headers/footers are not filtered, so the logical sequence of rows is polluted with navigation text.
  - **Impact**: Longitudinal comparisons (e.g., “trend of FTA scores across regions”) become unreliable because row ordering is broken and row keys are duplicated or missing.
- **Class D – Tax Expenditure Report (table-heavy structured data)**

- **Symptom**: Multi-year fiscal tables with hierarchical category labels (e.g., “Customs Duty → Category → Subcategory”) are flattened into poorly separated lines.
- **Root cause**:
  - Multi-row headers and hierarchical labels are laid out visually but not encoded as a true table object.
  - Without bounding boxes or grid detection, the system cannot reconstruct which header cell applies to which numeric cell.
- **Impact**: Queries like “Tax expenditure on import duty for fuel in 2019/20” return either the wrong row or a single ambiguous number with no contextual dimensions.

1.2 Context Poverty

- **Class A – CBE Annual Report**
  - **Symptom**: Token-based chunking (e.g., fixed 512–1024 tokens) splits:
    - Financial tables mid-row or between header and data.
    - Narrative discussion paragraphs away from the tables they reference.
  - **Root cause**:
    - Chunking by token count ignores document semantics and layout.
    - Figure/table references (“see Table 4.3”) are separated from the referenced table region.
  - **Impact**: RAG answers about “capital adequacy ratios” or “liquidity risk” may focus on the wrong table or misinterpret numbers due to missing header context.
- **Class B – DBE Audit Report (scanned)**
  - **Symptom**: OCRed text is noisy; if chunked naively, signature pages, audit opinion, and basis of opinion paragraphs are split across chunks.
  - **Root cause**:
    - OCR artifacts (line breaks, hyphenation, misrecognized characters) inflate token counts unevenly.
    - The logical unit “Audit opinion + basis” spans multiple physical lines and sometimes multiple pages.
  - **Impact**: Questions like “What is the auditor’s opinion on going concern?” are answered with partial clauses or incomplete rationale because the model only sees fragments.
- **Class C – FTA Technical Assessment**
  - **Symptom**: Findings sections (e.g., “Key weaknesses in internal controls”) are split such that bullet lists are scattered across chunks.
  - **Root cause**:
    - Numbered/bulleted lists are treated as separate paragraphs by naive splitters.
    - Section headers are detached from their subordinate bullets.
  - **Impact**: When asked “List the key weaknesses in cash management,” the model returns incomplete or mixed bullets from multiple sections.

1.3 Provenance Blindness

- **Across all classes (A–D)**
  - **Symptom**: Systems return numeric answers (e.g., revenue, tax expenditure, FTA scores) without **page number or bounding box**.
  - **Root cause**:
    - Pipelines store only raw text or embeddings; they discard spatial coordinates from pdfplumber or layout models.
    - No `content_hash` or stable spatial key is carried through from extraction to query-time.
  - **Impact**:
    - A reviewer cannot jump from an answer back to "the exact cell in the original PDF."
    - In regulated domains (banks, government), this breaks auditability—answers cannot be defended or reverified later.

---

2. Extraction Strategy Decision Tree (with Corpus Context)

The triage agent computes a `DocumentProfile` with:

- `origin_type`: `native_digital | scanned_image | mixed | form_fillable`
- `layout_complexity`: `single_column | multi_column | table_heavy | figure_heavy | mixed`
- `domain_hint`: `financial | legal | technical | medical | general`
- plus signals: char density, image area ratio, estimated tables per 10 pages, etc.

2.1 Routing Rules (Stage 1 → Stage 2)

- **If `origin_type = scanned_image` (e.g., Class B – DBE Audit Report)**
  - → **Strategy C (Vision)**
  - Rationale: character stream is absent or unreliable; pdfplumber yields near-zero chars and high image area ratio.
- **Else if `origin_type = native_digital` AND `layout_complexity = single_column` (e.g., simple narrative PDFs or some appendices in Class C)**
  - → **Strategy A (Fast Text)**
  - Rationale: fast, cheap, and adequate when there is a clean text stream with relatively uniform left margin.
- **Else (multi-column, table-heavy, or mixed layouts; typical for Classes A, C, D)**

    - → **Strategy B (Layout-Aware)**
    - Rationale: requires bounding boxes, table grid detection, and reading-order reconstruction to preserve structure.

2.2 Escalation Guard (A → B → C)

- After Strategy A runs on a document:
    - Compute **document-level confidence** from page-level signals (chars, char density, image ratio).
    - If `doc_confidence < min_doc_confidence_fast_text` (from `extraction_rules.yaml`):
        - **Automatic escalation A → B** (Layout).
    - If the **triage profile** indicates `origin_type = scanned_image` but Strategy A/B were used (e.g., misclassification or forced strategy):
        - **Escalation to C (Vision) regardless of confidence**, because any non-vision extraction is structurally untrustworthy.

Corpus mapping:

- **Class A/C/D**:
    - Many pages: `native_digital + multi_column/table_heavy` → Strategy B directly.
    - Some narrative sections (simple text) could stay on Strategy A, but the escalation guard ensures that if tables are poorly captured, they re-run under Strategy B.
- **Class B**:
    - `scanned_image` → Strategy C directly; Strategy A/B are fallback only if vision is disabled.

---

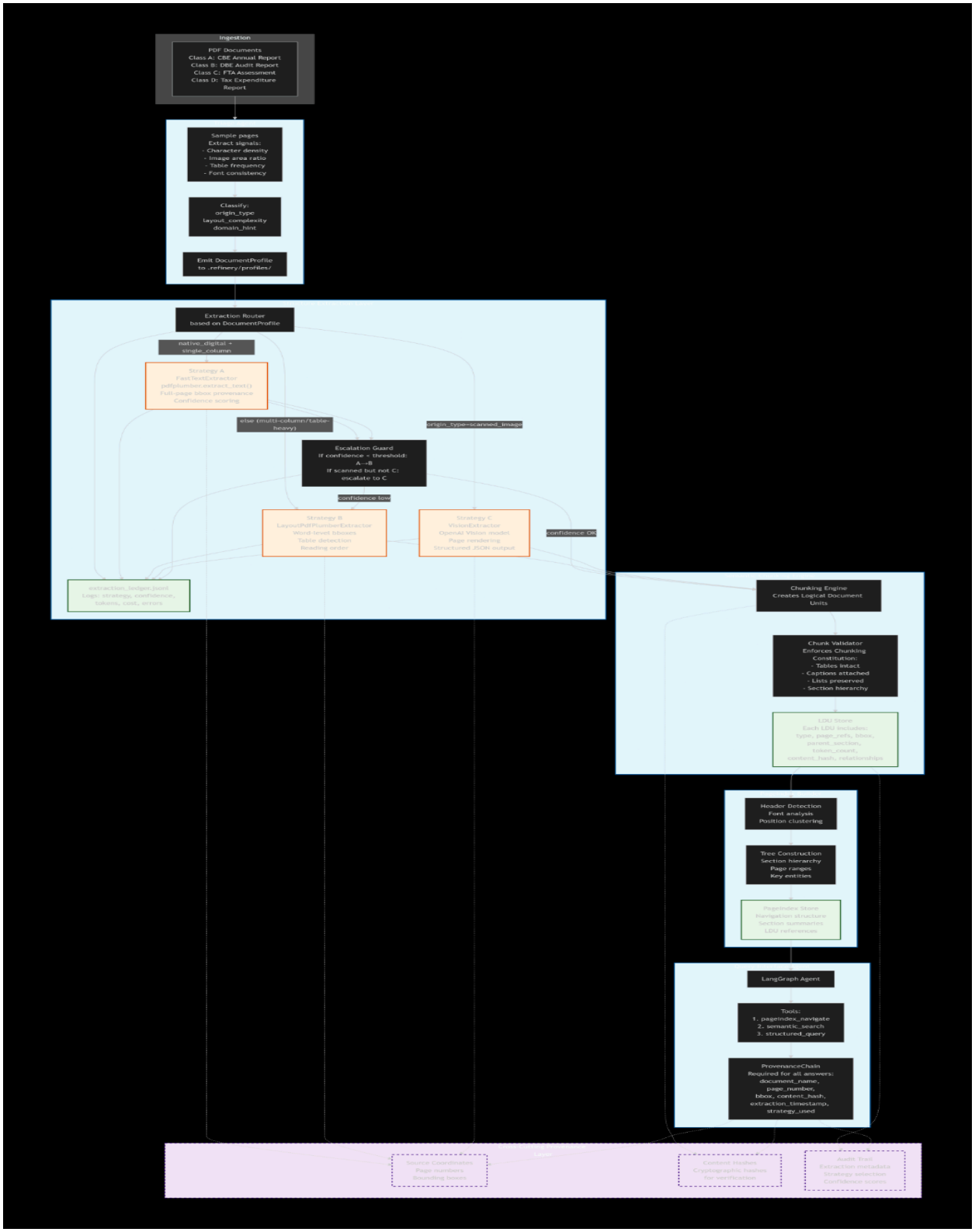
3. Architecture Diagram — Full 5-Stage Pipeline

3.1 Textual Overview

1. **Stage 1 — Triage Agent (`src/agents/triage.py`)**
    - Reads basic layout signals (char density, image area ratio, tables per 10 pages).
    - Detects origin type, layout complexity, domain hint, and recommended cost tier.
    - Emits `DocumentProfile` to `.refinery/profiles/{doc_id}.json`.
2. **Stage 2 — Structure Extraction Layer (`src/agents/extractor.py` + `src/strategies/*`)**
    - Applies Strategy A/B/C with **escalation guard**:
        - A: `FastTextExtractor` (pdfplumber text + page bbox provenance).

        - B: `LayoutPdfPlumberExtractor` (words + `find_tables()` with bounding boxes).
        - C: `VisionExtractor` (OpenAI VLM, JSON-only output).
    - Emits normalized `ExtractedDocument` JSON to `.refinery/extractions/{doc_id}.json`.
    - Logs strategy choice, confidence, and cost to `.refinery/extraction_ledger.jsonl`.
3. **Stage 3 — Semantic Chunking Engine (`src/agents/chunker.py`, planned)**
    - Consumes `ExtractedDocument` and emits **Logical Document Units (LDUs)**:
        - Each LDU carries `chunk_type`, `page_refs`, `bounding_box`, `parent_section`, `token_count`, and `content_hash`.
    - Enforces the "chunking constitution":
        - Table cell never split from header row.
        - Figure caption attached to figure chunk.
        - Numbered list is a single LDU unless it exceeds max tokens.
        - Section headers become parent metadata for all child LDUs.
        - Cross-references recorded as explicit relationships between LDUs.
4. **Stage 4 — PageIndex Builder (`src/agents/indexer.py`, planned)**
    - Builds a hierarchical `PageIndex` tree over sections/subsections:
        - Nodes contain: title, page_start/end, key_entities, presence of tables/figures/equations, and a short LLM summary.
    - This acts as a smart ToC so retrieval agents can:
        - First localise the **right section**.
        - Then search within the LDUs belonging to that section.
5. **Stage 5 — Query Interface Agent (`src/agents/query_agent.py`, planned)**
    - LangGraph agent with three tools:
        - `pageindex_navigate`: walk the `PageIndex` tree.
        - `semantic_search`: vector retrieval over LDUs.
        - `structured_query`: SQL over a fact table (e.g., financials in Class A/D).
    - Every answer must return a **ProvenanceChain**:
        - `document_name`, `page_number`, `bbox`, `content_hash`.

3.2 Mermaid Diagram (with Escalation + Provenance Layer)



The **dashed Provenance Layer** indicates that page coordinates, bounding boxes, and `content_hash` flow through all stages and are surfaced in Stage 5 responses.

---

4. Cost Analysis — Per-Document Estimates by Strategy Tier

Assumptions (order-of-magnitude; adjust with real token counts and OpenAI pricing):

- Average page tokenization:
  - **Text-only page** (Classes A, C narrative): ~500–800 tokens.
  - **Table-heavy page** (Class D): ~600–1000 tokens once serialized as JSON.
  - **Vision page** (image + structured JSON output): ~1500–2500 “equivalent tokens” (prompt + completion).
- Price example (illustrative, not authoritative):
  - `gpt-4.1-mini` text:
    - (p_\text{in} \approx $0.00015 / 1K) tokens
    - (p_\text{out} \approx $0.00060 / 1K) tokens
  - Vision-capable model (OpenAI Vision tier):
    - (p_\text{in} \approx $0.00060 / 1K) tokens
    - (p_\text{out} \approx $0.00120 / 1K) tokens

4.1 Strategy A — Fast Text (Low Cost)

- **Workload**:
  - `pdfplumber` text extraction only; no LLM calls.
- **Typical documents**:
  - Short narrative PDFs or simple sections of Class C (appendices, letters).
- **Cost per document**:
  - **API cost**: ~$0.00 (pure local computation).
  - **CPU time**: O(0.1–1.0 s) for 20–100 pages on a typical laptop.
- **Quality trade-off**:
  - Excellent for **plain, single-column text** (e.g., executive summary).
  - Poor for multi-column layouts or dense tables (Classes A, D) → triggers escalation if confidence is low.

4.2 Strategy B — Layout-Aware (Medium Cost)

- **Workload**:
  - `pdfplumber` word extraction + `find_tables()` + bounding boxes.

    - Optional cheap LLM calls later for summarization (Stages 3–4), not for raw extraction.
- **Typical documents**:
    - Class A (CBE Annual Report) — most pages.
    - Class C (FTA Report) — narrative + embedded tables.
    - Class D (Tax Expenditure) — table-heavy content.
- **Cost per document**:
    - **API cost** (extraction only): $0.00 (still local).
    - **CPU time**:
        - 100-page Class A or C document: ~5–20 s depending on number of tables detected.
        - 200-page Class D document: ~15–40 s with heavy `find_tables()` usage.
- **Quality trade-off**:
    - Much higher **table fidelity** and structure preservation (rows, headers, cell grouping).
    - Slightly higher runtime and engineering complexity, but still free from an API perspective.

4.3 Strategy C — Vision-Augmented (High Cost)

- **Workload**:
    - Render page images (PyMuPDF).
    - Call OpenAI Vision with prompt + page images; receive structured JSON with `text_blocks` and `tables`.
- **Typical documents**:
    - Class B (DBE Audit Report) — pure scanned images.
    - Any pages in A/C/D where:
        - Digital extraction confidence is low, or
        - Handwriting, stamps, or highly graphical tables dominate.
- **Cost per document** (example: 50-page scanned PDF, first 10 pages processed by Vision):
    - Assume 2000 tokens per page (prompt + completion) on a Vision model:
        - **Input**: (10 \times 1500 = 15,000) tokens → (15 \times p_\text{in}).
        - **Output**: (10 \times 500 = 5,000) tokens → (5 \times p_\text{out}).
    - With (p_\text{in} = 0.00060), (p_\text{out} = 0.00120) per 1K tokens:
        - Input cost: (15 \times 0.00060 = $0.009).
        - Output cost: (5 \times 0.00120 = $0.006).
        - **Total ≈ $0.015 per 10 pages** → ~$0.075 for 50 pages (order of magnitude).
    - For a full 200-page scanned report, naïve Vision on all pages might reach **$0.20–$0.30+** per document.
- **Quality trade-off**:
    - Recovers structure even when **no digital text** exists.

    - Handles complex tables with borders, stamps, and non-standard fonts better than OCR alone.
    - Must be used **selectively** (via triage + budget guard) to keep costs under control.

4.4 Comparative Summary by Class

- **Class A (Annual Report)**:
    - Strategy B for most pages; A can be used on simple narrative sections.
    - **Cost**: essentially zero API cost, moderate CPU (~10–20 s).
    - **Quality**: high-fidelity tables + section text, sufficient for financial Q&A with provenance.
- **Class B (Scanned Audit Report)**:
    - Strategy C on key sections (opinion, basis, main financial statements), not necessarily every page.
    - **Cost**: on the order of **$0.02–$0.10 per document**, depending on how many pages are escalated.
    - **Quality**: ability to answer legal/financial questions with a verifiable link back to the scanned image.
- **Class C (FTA Technical Assessment)**:
    - Strategy B as default; occasional C for poor-quality charts or scanned annexes.
    - **Cost**: mostly CPU-bound; Vision cost only where necessary.
    - **Quality**: preserved section hierarchy, tables, and bullet lists for nuanced policy questions.
- **Class D (Tax Expenditure Report)**:
    - Strategy B for most content; C only if some tables are embedded as raster images.
    - **Cost**: similar to Class A/C, dominated by CPU; Vision cost rare but justified for critical numeric tables.
    - **Quality**: reliable extraction of multi-year numeric tables, enabling SQL-style joins and precise analytics.

These estimates justify having a configurable `max_usd_per_document` in `rubric/extraction_rules.yaml`: for example, cap Strategy C at **$0.10 per document** in a pilot, then adjust once empirical usage data is available.